Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Extadas e Informática
Algoritmos e Estruturas de Dados II

Trabalho Prático II

Regras Básicas

1. extends Trabalho Prático 01

2. Neste trabalho, nas questões em Java, adicionalmente podem ser utilizados os métodos `int compareTo(String str)` e `static String format(String str, Object... args)` da classe `String`. Nas questões em C, adicionalmente podem ser utilizadas as funções `int strcmp(char* str1, char* str2)` de `string.h`, `int sscanf(char* str, char* format, ...)` e `int sprintf(char* buffer, char* format, ...)` de `stdio.h`

Dataset - Restaurantes pelo Mundo

Este dataset apresenta uma coleção diversificada de restaurantes distribuídos por diferentes cidades ao redor do mundo, refletindo a riqueza cultural e gastronômica de diversas regiões. A base reúne estabelecimentos com diferentes perfis, variando em capacidade, faixa de preço, tipos de cozinha e avaliações, permitindo uma visão ampla do cenário gastronômico global.

Através desses dados, é possível explorar padrões interessantes, como a relação entre localização e estilo culinário, variações de avaliação entre diferentes tipos de restaurantes e características operacionais como horário de funcionamento e tempo de atividade. Cada registro representa um estabelecimento único, contribuindo para um conjunto de dados rico e multifacetado.

Este dataset convida à investigação e análise, estimulando a descoberta de tendências, comparações entre cidades e o desenvolvimento de soluções que simulam aplicações do mundo real, como sistemas de busca, recomendação e organização de informações.

O dataset permite explorar conceitos importantes como organização e tipagem de dados, modelagem de entidades do mundo real, manipulação de estruturas (listas, registros, etc.), bem como consultas, filtragens e ordenações.

A tabela a seguir apresenta o dicionário de dados referente ao dataset. O dataset está disponível no arquivo RESTAURANTES.CSV.

Campo	**Descricao**	**Tipo / Formato**
id	Identificador unico do restaurante (chave primaria)	Inteiro (int)
nome	Nome do restaurante	String (texto ASCII)
cidade	Cidade onde o restaurante esta localizado	String (texto ASCII)
capacidade	Numero maximo de clientes que o restaurante comporta	Inteiro (int)
avaliacao	Nota media do restaurante (simulando avaliacoes de usuarios)	Numero real (double, ex: 4.5)
tipos_cozinha	Tipos de culinaria oferecidos (pode conter mais de um tipo)	Lista (string separada por ;)
faixa_preco	Nivel de preco do restaurante	String ($, $$, $$$, $$$$)
horario	Horario de funcionamento diario	String (HH:mm-HH:mm)
data_abertura	Data de abertura do restaurante	Data (YYYY-MM-DD)
aberto	Indica se o restaurante esta em funcionamento atualmente	Booleano (true ou false)

Tabela 1: Dicionario de dados do dataset de restaurantes

Classe + Registro

O Verde pressupõe a presença do arquivo do dataset (RESTAURANTES.CSV) na pasta /tmp/, de onde seu programa fará a leitura. Portanto, o arquivo do dataset deve ser copiado para a pasta /tmp/ em seu ambiente Linux. Quando reiniciamos o Linux, ele normalmente apaga os arquivos existentes na pasta /tmp/.

Esses tipos devem ser definidos de forma clara e organizada, permitindo representar fielmente os dados provenientes de um arquivo CSV contendo informações sobre restaurantes.

Modelagem de Dados: Restaurante

Neste trabalho, voce deverá realizar a modelagem de dados de um sistema que representa uma coleção de restaurantes distribuídos em diferentes cidades do mundo.

O objetivo e definir estruturas de dados que permitam representar adequadamente as informações de cada restaurante, servindo como base para etapas futuras do trabalho, nas quais serao aplicados algoritmos e estruturas de dados como listas, filas, árvores, além de operacões de pesquisa e ordenação.

Objetivo

Projetar e implementar os seguintes tipos de dados:

- Restaurante
- Data
- Hora

Esses tipos devem ser definidos de forma clara e organizada, permitindo representar fielmente os dados provenientes de um arquivo CSV contendo informações sobre restaurantes.

Estilo de codificação

Para a codificação, deve-se utilizar as seguintes convenções de nomenclatura:

- Java: camel case
- C: snake case

Tipos (*class* e *struct*) devem iniciar com letra maiúscula, enquanto variáveis e métodos (funções) devem iniciar com letra minúscula.

Especificação dos Tipos

Tipo Data

Representa uma data no calendário.

- ano (inteiro)
- mes (inteiro)
- dia (inteiro)

Tipo Hora

Representa um horário do dia.

- hora (inteiro, de 0 a 23)
- minuto (inteiro, de 0 a 59)

Tipo Restaurante

Representa um restaurante com suas características principais.

- id: identificador unico (inteiro)
- nome: nome do restaurante (string)

- cidade: cidade onde esta localizado (string)
- capacidade: capacidade maxima de clientes (inteiro)
- avaliacao: nota media (double)
- tipos_cozinha: colecao de tipos de culinaria (arranjo/lista de strings)
- faixa_preco: nivel de preco (inteiro de 1 a 4), onde:
  - 1 → $
  - 2 → $$
  - 3 → $$$
  - 4 → $$$$
- horario_abertura: horario de abertura (tipo Hora)
- horario_fechamento: horario de fechamento (tipo Hora)
- data_abertura: data de abertura (tipo Data)
- aberto: indica se o restaurante esta em funcionamento (booleano)

Regras de Mapeamento (CSV → Estruturas)

Ao ler o arquivo CSV:

- O campo `tipos_cozinha` deve ser convertido em um arranjo de strings, utilizando o separador `;`
- O campo `faixa_preco` deve ser convertido para um inteiro de 1 a 4
- O campo `horario` (formato `HH:mm-HH:mm`) deve ser dividido em:
  - `horario_abertura`
  - `horario_fechamento`
- O campo `data_abertura` (formato `YYYY-MM-DD`) deve ser convertido para o tipo Data
- O campo `aberto` deve ser interpretado como valor booleano

Coleção de Restaurantes

Estrutura que representa uma coleção de restaurantes para armazenar os restaurantes lidos do dataset. Essa coleção deve ser modelada como um arranjo de ponteiros para Restaurante e será responsável por ler o arquivo CSV. Crie um tipo denominado *ColecaoRestaurantes* com os campos:

- tamanho: quantidade de itens (inteiro)
- restaurantes (arranjo de ponteiros para Restaurante)

Observações

- A modelagem deve ser compatível tanto com linguagem C quanto Java
- É vedada a dependência de bibliotecas externas
- Priorize clareza, organização e boa definição dos tipos
- Pense na reutilização das estruturas em etapas futuras

Próximos Passos

Em etapas futuras, essa modelagem será utilizada para:

- leitura completa do dataset
- ordenação por diferentes critérios (nome, avaliacao, cidade)
- busca (sequencial e binária)
- uso de estruturas como pilhas, filas e árvores

Exercícios

1. **Modelagem em Java**: Crie os tipos solicitados seguindo todas as regras apresentadas no slide unidade00l_conceitosBasicos_introducaoOO.pdf. Lembre-se:

   - os tipos são criados usando classes
   - atributos devem ser privados (encapsulamento!!!)
   - implemente os métodos *get* para os atributos
   - avalie se deve ou não implementar os métodos *set* para os atributos
   - em Java, use padrão de nomenclatura camel case

   .

   Os seguintes métodos devem estar presentes para os tipos:

   - Data
     - public static Data parseData(String s): recebe uma string que representa uma data no formato YYYY-MM-DD e retorna um novo objeto Data correspondente
     - public String formatar(): retorna uma string que representa a data no formato DD/MM/YYYY
   - Hora
     - public static Hora parseHora(String s): recebe uma string que representa uma hora no formato HH:mm e retorna um novo objeto Hora correspondente

  - public String formatar(): retorna uma string que representa a hora no formato HH:mm

- Restaurante
  - public static Restaurante parseRestaurante(String s): recebe uma string que representa um restaurante no mesmo formato do dataset e retorna um novo objeto Restaurante correspondente
  - public String formatar(): retorna uma string que representa o restaurante no seguinte formato: `[=> id ## nome ## cidade ## capacidade ## avaliacao ## [tipos_cozinha] ## faixa_preco ## horario_abertura-horario_fechamento ## data_abertura ## aberto]`

    Exemplo:

    `[478 ## Le Gourmet 478 ## Paris ## 178 ## 4.8 ## [francesa,bistro] ## $$$ ## 12:00-20:00 ## 12/07/2020 ## false]`

- ColecaoRestaurantes
  - public int getTamanho(): retorna a quantidade de restaurantes da coleção
  - public Restaurante[] getRestaurantes(): retorna o arranjo com os restaurantes (ponteiros) da coleção
  - public void lerCsv(String path): lê o arquivo CSV, cria os restaurantes e configura a coleção
  - public static ColecaoRestaurantes lerCsv(): lê o dataset do arquivo CSV e retorna a coleção com os restaurantes

Nessa questão, a entrada de dados (entrada padrão) contém um conjunto de números inteiros organizados um por linha, sendo que cada número representa o `id` de um restaurante. A última linha da entrada de dados contém -1, indicando o fim.

Seu programa devrá imprimir na saída padrão, em cada linha, o restaurante correspondente ao `id` lido na entrada.

2. **Modelagem em C**: Repita o exercício de modelagem anterior, agora na linguagem C. Lembre-se:

   - os tipos são criados usando structs
   - em C, use padrão de nomenclatura snake case

   Realizando as adaptações, as seguintes funções devem estar presentes para os tipos:

   - Data
     - Data parse_data(char* s): recebe uma string que representa uma data no formato YYYY-MM-DD e retorna uma nova instância de Data correspondente

  - void formatar_data(Data* data, char* buffer): recebe o ponteiro para a data e escreve no buffer a string que representa a data no formato DD/MM/YYYY (dica: use `sprintf`)

- Hora
  - Hora parse_hora(char* s): recebe uma string que representa uma hora no formato HH:mm e retorna uma nova instância de Hora correspondente
  - void formatar_hora(Hora* hora, char* buffer): recebe o ponteiro para a hora e escreve no buffer a string que representa a hora no formato HH:mm

- Restaurante
  - Restaurante* parse_restaurante(char* s): recebe uma string que representa um restaurante no mesmo formato do dataset e retorna um ponteiro para a nova instância Restaurante correspondente
  - void formatar_restaurante(Restaurante* restaurante, char* buffer): recebe o ponteiro para o restaurante e escreve no buffer a string que representa o restaurante no formado informado anteriormente

- Colecao_Restaurantes
  - void ler_csv_colecao(Colecao_Restaurantes* colecao, char* path): lê o arquivo CSV, cria os restaurantes e configura a coleção
  - ColecaoRestaurantes* ler_csv(): lê o dataset do arquivo CSV e retorna a coleção (ponteiro) com os restaurantes

Pesquisa e Ordenação

Para os exercícios de pesquisa, seu programa deve criar um arquivo de log na pasta corrente com o nome `matrícula_<algoritmo>.txt` com uma única linha contendo sua matrícula, número de comparações de todas as pesquisas e tempo de execução total das pesquisas. Todas as informações do arquivo de log devem ser separadas por uma tabulação '\t'.

Para os exercícios de ordenação, seu programa deve criar um arquivo de log na pasta corrente com o nome `matrícula_<algoritmo>.txt` com uma única linha contendo sua matrícula, número de comparações (entre elementos do *array*), número de movimentações (entre elementos do *array*) e o tempo de execução do algoritmo de ordenação. Todas as informações do arquivo de log devem ser separadas por uma tabulação '\t'.

3. **Ordenação por Seleção em C**: Usando arranjos, implemente o algoritmo de ordenação por seleção considerando que a chave de pesquisa é o atributo **nome**. A entrada e a saída padrão

são iguais as da primeira questão, contudo, a saída corresponde aos registros ordenados. Nome do arquivo de log: `matrícula_selecao.txt`.

4. **Ordenação por Inserção em Java**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo de Inserção. A chave de pesquisa é o atributo **cidade**. Nome do arquivo de log: `matrícula_insercao.txt`.

5. **Pesquisa Sequencial em Java**: Implemente o algoritmo de pesquisa sequencial considerando a chave primária de pesquisa o atributo **nome**. A entrada padrão é composta por duas partes onde a primeira é igual a entrada da primeira questão. O conjunto de restaurantes da primeira parte deve ser armazenado em um arranjo separado, que servirá de base para as pesquisas sequenciais a serem realizadas. A segunda parte da entrada de dados é composta por várias linhas. Cada uma possui um elemento que deve ser pesquisado no arranjo. A última linha terá a palavra FIM. A saída padrão será composta por várias linhas contendo as palavras SIM/NAO para indicar se existe cada um dos elementos pesquisados. Nome do arquivo de log: `matrícula_sequencial.txt`.

6. **Pesquisa Binária em C**: Implemente o algoritmo de pesquisa binária considerando a chave primária de pesquisa o atributo **nome**. A entrada e a saída padrão serão iguais as da questão anterior. Para a ordenação, use o algoritmo de seleção implementado anteriormente. Nome do arquivo de log: `matrícula_binaria.txt`.

7. **Ordenação por Mergesort em Java**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo Mergesort, fazendo com que a chave de pesquisa seja o atributo **cidade**. Em caso de empate, o critério de ordenação deverá ser o **nome**. Nome do arquivo de log: `matrícula_mergesort.txt`.

8. **Ordenação por Quicksort em C**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo Quicksort, fazendo com que a chave de pesquisa seja o atributo **avaliacao**. Em caso de empate, o critério de ordenação deverá ser o **nome**. Nome do arquivo de log: `matrícula_quicksort.txt`.

9. **Ordenação por Heapsort em Java**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo Heapsort, fazendo com que a chave de pesquisa seja o atributo **data_abertura**. Em caso de empate, o critério de ordenação deverá ser o **nome**. Nome do arquivo de log: `matrícula_heapsort.txt`.

10. **Ordenação por Counting Sort em C**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo Counting Sort, fazendo com que a chave de pesquisa seja o atributo **capacidade**. Nome do arquivo de log: `matrícula_countingsort.txt`.

Estruturas Sequenciais

11. **Lista com Alocação Sequencial em Java**: Crie uma Lista de registros baseada na de inteiros vista na sala de aula. Sua lista deve conter todos os atributos e métodos existentes na lista de inteiros, contudo, adaptados para a classe `Restaurante`. Lembre-se que, na verdade, temos uma lista de ponteiros (ou referências) e cada um deles aponta para um registo. Neste exercício, faremos inserções, remoções e mostraremos os elementos de nossa lista.

    Os métodos de inserir e remover devem operar conforme descrito a seguir, respeitando parâmetros e retornos. Primeiro, o `void inserirInicio(Restaurante restaurante)` insere um registro na primeira posição da Lista e remaneja os demais. Segundo, o `void inserir(Restaurante restaurante, int posicao)` insere um registro na posição *posicao* da Lista, onde $posicao < n$ e n é o número de registros cadastrados. Em seguida, esse método remaneja os demais registros. O `void inserirFim(Restaurante restaurante)` insere um registro na última posição da Lista. O `Restaurante removerInicio()` remove e retorna o primeiro registro cadastrado na Lista e remaneja os demais. O `Restaurante remover(int posicao)` remove e retorna o registro cadastrado na posição informada da Lista e remaneja os demais. O `Restaurante removerFim()` remove e retorna o último registro cadastrado na lista.

    A entrada padrão é composta por duas partes. A primeira parte é igual a entrada da primeira questão e os registros presentes devem ser inseridos ao final da Lista. As demais linhas correspondem a segunda parte. A primeira linha da segunda parte é composta de um número inteiro n indicando a quantidade de registros a serem inseridos/removidos. Nas próximas n linhas, tem-se n comandos de inserção/remoção a serem processados neste exercício. Cada uma dessas linhas tem uma palavra de comando: `II` inserir no início, `I*` inserir na posição informada, `IF` inserir no fim, `RI` remover no início, `R*` remover na posição informada e `RF` remover no fim. No caso dos comandos de inserção, temos também o identificador do registro a ser inserido. No caso dos comandos de inserção na posição informada, a posição fica imediatamente após a palavra de comando. A saída padrão tem uma linha para cada registro removido, sendo que essa informação será constituída pela palavra "(R)" e o atributo **nome** do registro removido. No final, a saída mostra os registros presentes na Lista do primeitro ao último.

12. **Pilha com Alocação Sequencial em C**: Crie uma Pilha de registros baseada na pilha de inteiros vista na sala de aula. Neste exercício, faremos inserções, remoções e mostraremos os elementos de nossa pilha a partir do topo. A entrada e a saída padrão serão como as da questão Lista com Alocação Sequencial, contudo, teremos apenas os comandos `I` para inserir (empilhar) e `R` para remover (desempilhar).

13. **Fila Circular com Alocação Sequencial em Java**: Crie *Fila Circular* de registros.

Essa fila deve ter tamanho cinco. A entrada padrão será como a da questão Lista com Alocação Sequencial, contudo, teremos apenas os comandos `I` para inserir (enfileirar) e `R` para remover (desenfileirar). Quando o programa tiver que inserir um registro e a fila estiver cheia, antes, ele deve fazer uma remoção.

Na saída padrão deverá constar, para cada inserção, a palavra "(I)" seguida de um número inteiro que corresponde à média **arredondada do ano de abertura** dos registros contidos na fila após a inserção. Além disso, para cada registro removido da fila, a saída padrão também apresenta a palavra "(R)" e o atributo **nome** do registro removido. No final, a saída mostra os registros presentes na fila do primeitro ao último.